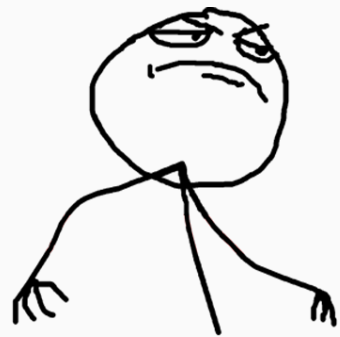


`jq`: deal with `JSON` like a boss

Xavier "Pamplemousse" Maso

2026-05-31



JdLLSON

🍊 Xavier Maso (il/lui)

🔧 RSSI en prestation au [Laboratoire de l’Innovation de l’ANSSI](#)

💻 git.sr.ht/~pamplemousse¹

👤 mamot.fr/@Pamplemouss_

🏠 www.xaviermaso.com



¹Vous y trouverez d’ailleurs les slides, et l’environnement utilisé pour les démos : git.sr.ht/~pamplemousse/jq_like_a_boss.

—

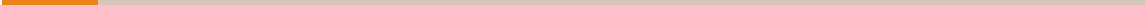
Indépendant, dans une coopérative d’entrepreneurs.

En mission au Laboratoire d’Innovation de l’ANSSI, incubateur membre du réseau BetaGouv.

- 1 Intro 3
- 2 Pré-requis... JSON 6
- 3 101 10
 - 3.1 Lancer jq, filtres 11
 - 3.2 Extraire des informations 12
 - 3.3 Chainer des opérations 14
 - 3.4 Opérations 15
 - 3.5 Produire des valeurs 16
- 4 Boucler 17
 - 4.1 Avec des objets 19
 - 4.2 Avec des chaines de caractères 20
 - 4.3 Trier et agréger 21
 - 4.4 Prédicats 22
- 5 CLI 23
- 6 Programmation (fonctionnelle) 25
 - 6.1 Transformer 26
 - 6.2 Fonctions 27
 - 6.3 Variables 28
- 7 Autres trucs utiles 29
 - 7.1 Astuces 30
 - 7.2 Aller trop loin 31
- 8 Quelques exemples 32
- 9 Crédits 35

—

1 Intro



jq is like sed for JSON data - you can use it to slice and filter and map and transform structured data with the same ease that sed, awk, grep and friends let you play with text.

[...]

jq can mangle the data format that you have into the one that you want with very little effort, and the program to do so is often shorter and simpler than you'd expect.

— <https://jqlang.org/>

—

manuels

- <https://jqlang.org/manual>
- <https://manpages.org/jq>

playground

- <https://play.jqlang.org>

s'entraîner

- <https://exercism.org/tracks/jq>

manuels

- <https://jqlang.org/manual>
- <https://manpages.org/jq>

playground

- <https://play.jqlang.org>

s'entraîner

- <https://exercism.org/tracks/jq>



fin de la prés, merci, de rien, au revoir messieurs-dames

— ~ Perceval

—

2 Pré-requis... JSON

2 Pré-requis... JSON

—

JSON (JavaScript Object Notation) is a lightweight data-interchange format. It is easy for humans to read and write. It is easy for machines to parse and generate. It is based on a subset of the JavaScript Programming Language

— <https://www.json.org/>

—

```
1 {
2   "name": {
3     "last": "Maso",
4     "first": "Xavier",
5     "handle": "Pamplemousse"
6   },
7   "yearOfBirth": 1990,
8   "opinions": {
9     "fanOf": ["jq", "elm", "nix"],
10    "hatesAI": true
11  },
12  "profiles": [{
13    "network": "Mastodon",
14    "url": "https://mamot.fr/@pamplemouss_"
15  }]
16 }
```

—

2 structures :

- collection de paires clef-valeurs (objet, record, stuct, dictionnaire, hashed table, etc.)
- liste ordonnée (tableau, vecteur, liste, séquence)

objet, tableau, chaine, nombre, “true”, “false”, “null”

récuratif

Des sous-ensembles intéressants et/ou amusants :

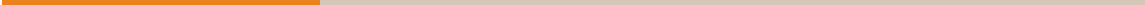
- [geoJSON](#) : encodage de données géospatiales,
- [SARIF](#) : format standardisé de sortie d'outils d'analyse statique de code,
- [jsonresume](#) : représentation des données d'un CV,
- [HCL JSON](#) : une syntaxe alternative pour de la configuration-as-code,
- [nix derivation show <store path>](#) : la représentation JSON des informations d'une dérivation nix,
- autres...

... et si on a pas de JSON 🐱 ?

- [jc](#) : outil en CLI qui permet de convertir les sorties de différents outils en CLI,
- [yq](#) : wrapper de jq pour gérer du YAML/XML/TOML.

—

3 101



3 101
—

```
< examples/pamplemousse.json jq '.'
```

by default, jq reads a stream of JSON entities (including numbers and other literals) from stdin

[...]

A jq program is a “filter”: it takes an input, and produces an output.

[...]

Filters can be combined in various ways

— man jq

identité	.
index de tableau	. [N]
index d'objet	.clef ["clef"]
slice	. [N:M]
valeur alternative	chemin // défaut
filtre les éléments d'un objet ou d'une liste	pick(un_chemin, ...)

“path expressions” complexes `.clef.autreClef.[0]`

	keys
les indices d'un objet	keys_unsorted
taille d'une liste	length

L’opérateur qui fera de vous un plombier heureux, avec un magnifique sourire.



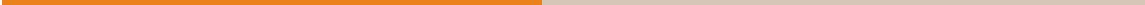
arithmétique	<code>+, -, *, /, %</code>
booléenne	<code>and, or, not</code>
comparaisons	<code>==, !=, <, >, <=, >=</code>
interpolation de chaines	<code>"... \ (filtre) ..."</code>

`jq '<value> | not`

Créer des...

	N
scalaires	" ... "
listes	[...]
objets	{ ... }

4 Boucler



appliquer un filtre aux éléments d'une liste	<code>map(f)</code>
appliquer un filtre aux valeurs d'un objet	<code>map_values(f)</code>
désimbriquer des listes de listes	<code>flatten</code>
filtrer une valeur	<code>select(b)</code>
aggréger	<code>reduce .[] as \$e (valeur; f)</code>

—

Et donc, pour filtrer une liste :

```
map(select(b))
```

```
echo '{"a": "b", "c": "d"}' | jq 'with_entries(.key |= "_" + . | .value |= "_" + .)'
```

transformer un objet en liste	to_entries
transformer une liste en objet	from_entries
to_entries map(f) from_entries	with_entries(f)

découper une chaine de caractères en liste	<code>split(c)</code>
transformer une liste en chaine de caractères	<code>join(c)</code>

trier une liste	<code>sort</code>
trier un objet	<code>sort_by(f)</code>
	<code>unique</code>
dédupliquer les éléments d'une liste	<code>unique_by</code>

inclusion : pour les chaines de caractères, les listes, et les objets

inclusion	<code>contains(valeur)</code>
présence d'un champ dans l'objet	<code>has("clef")</code>
au moins une valeur doit être vraie	<code>any</code> <code>any(condition)</code>
tout doit être vrai	<code>all</code> <code>all(condition)</code>

5 CLI



5 CLI
—

sortie brute	- - raw-output / -r
sans pretty-print	- - compact-output / -c
charger plusieurs valeurs `JSON`	- - slurp / -s
tri les objets par clefs	- - sort-keys / -S
charger du code depuis un fichier	- - from-file / -f
ne pas attendre d'entrée	- - null-input / -n
	- - arg <name> <value>
passer une valeur labellisée	... \$<name> ...

—

```
$ (echo '{"name": "Pamplemousse"}'; sleep 2; echo '{"name":
"Xavier"}') | jq '.name'
"Pamplemousse"
"Xavier"
$ (echo '{"name": "Pamplemousse"}'; sleep 2; echo '{"name":
"Xavier"}') | jq --slurp 'map(.name)'
[
  "Pamplemousse",
  "Xavier"
]
```

6 Programmation (fonctionnelle)

	<code>del(.clef)</code>
suppression	<code>del(. [N])</code>
assignation	<code>chemin = valeur</code>
mise à jour	<code>chemin = expression</code>

Values in jq are always immutable. Internally, assignment works by [computing] replacement values for . that have had all the desired assignments applied to ., then outputting the modified value.

— man jq

|= est l’update assignment, et permet de mettre à jour en fonction de la valeur précédente

- nommer des filtres, et les réutiliser
- ~lazy : les arguments sont passés comme des filtres, pas comme des valeurs !

définition	<code>def nom_de_fonction: f;</code>
appel	<code>nom_de_fonction</code>
définition avec paramètre	<code>def nom_de_fonction(p): f;</code>
appel avec paramètre	<code>nom_de_fonction(v) f nom_de_fonction</code>

```
jq -nfr ./examples/functions.jq`
```

Généralement : pas besoin.

In jq, all filters have an input and an output, so manual plumbing is not necessary to pass a value from one part of a program to the next. Many expressions, for instance `a + b`, pass their input to two distinct subexpressions (here `a` and `b` are both passed the same input), so variables aren't usually necessary in order to use a value twice.

— `man jq`

The expression `exp as $x | ...` means: for each value of expression `exp`, run the rest of the pipeline with the entire original input, and with `$x` set to that value.

— `man jq`

7 Autres trucs utiles

rien (même pas `null`)	<code>empty</code>
<code>map(.clef select(. != null))</code>	<code>map(.clef // empty)</code>
concatène plusieurs filtres	<code>f1, f2</code>
formater la sortie en CSV	<code>@CSV</code>
gérer les dates	<code>fromdate,</code> <code>todateiso8601,</code> <code>fromdateiso8601,</code> <code>strftime</code>

exploration en profondeur	. .
lire le type d'une valeur	valeur type
expressions régulières	test, match, scan
gestion des erreurs	try-catch
streaming	- - stream
50 nuances de récursion	while, until, repeat, recurse, walk
Manipuler des « Path Expressions » 👨‍🍳🐱	path, getpath, setpath, delpaths

!= de l'évaluation dynamique

8 Quelques exemples

8 Quelques exemples

- 🍊 ./examples/pamplemousse.json
- 🚒 ./examples/s3_policy.json.json
- 🌳 ./examples/ec_parc_s.json
- 🧪 ./examples/functions.jq
- 🐙 ./examples/evenements_github/*.json, ./examples/evenements_github/main.jq

8 Quelques exemples

S3 policy

add

```
< examples/s3_policy.json jq --arg uuid 60b05fcb-7be6-4516-  
ba9d-846f081d09b1 '.Statement |= map(.Principal.AWS |= ([.,  
"arn:aws:iam:::user/\($uuid)"] | flatten))'
```

delete

```
< examples/s3_policy.json jq --arg uuid ec44a279-e4bf-49f2-  
a891-7df4b750c617 '.Statement |= map(.Principal.AWS |=  
map(select(scan("user/*.") != "user/\($uuid)")))'
```

Évènements GitHub

```
cat examples/evenements_github/*.json | jq --slurp --  
library-path ./examples/evenements_github 'include "./main";  
getPullRequestsContributions'
```

JdLL

9 Crédits



9 Crédits
—

- [typst](#) - Un système de composition de documents moderne, puissant, et facile à apprendre.
- [typix](#) - Compilation déterministe de Typst avec Nix.
- [touying](#) - Création de présentations en Typst.
- [Nix/NixOS](#) - Un gestionnaire de paquet, et de configuration système, déclaratif.
- [opendata.bordeaux-metropole.fr](#) - Des sources de données ouvertes avec lesquelles jouer.

—